

Linear Search Algorithm

0 1 2 3 4 5 size = 6
arr = [18, 12, 9, 14, 77, 50]
↳ unsorted

Q :- find whether 14 exists in array or not.
if no value found, return -1.

Time Complexity: means how your time going to grow as the input grows.

Best : $O(1)$ // constant

worst : $O(N)$

↳ N is the size of array

Best case:

* How many checks will the loop make in best case i.e. element found at 0th index?

i = 0
arr = [8, 9, 12, 18, ... 200 elements]

target = 8

→ 1 comparison made to reach the target so called best case.

- The number of Comparisons in the best case does not depend on the size of array b/c your item would be found at index 0th.

Best case = only 1 comparison

so size of the array doesn't matter.
that's why time complexity is constant in best case.

Worst Case Scenario:

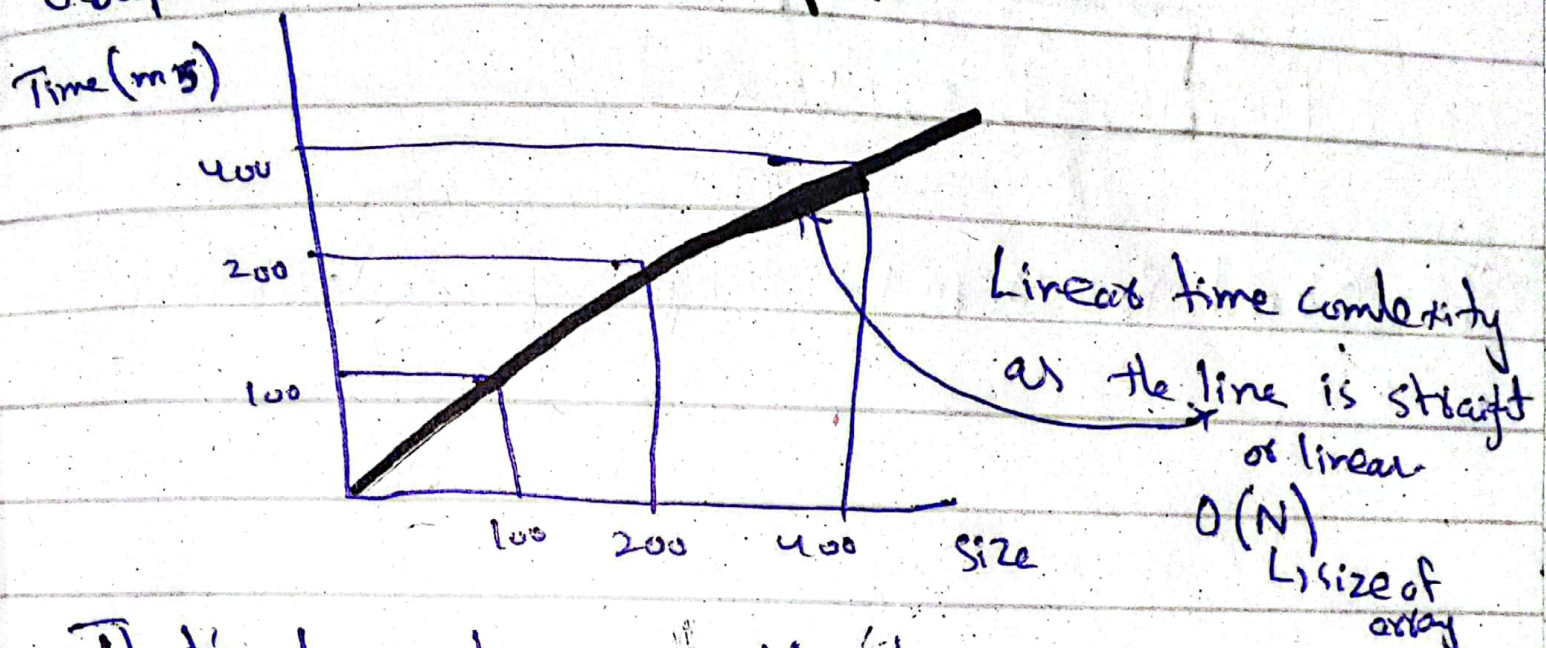
↳ you don't find the targeted item.

↳ checked through every iteration & compare but do not find the one.

In end, it says, i didn't find it.

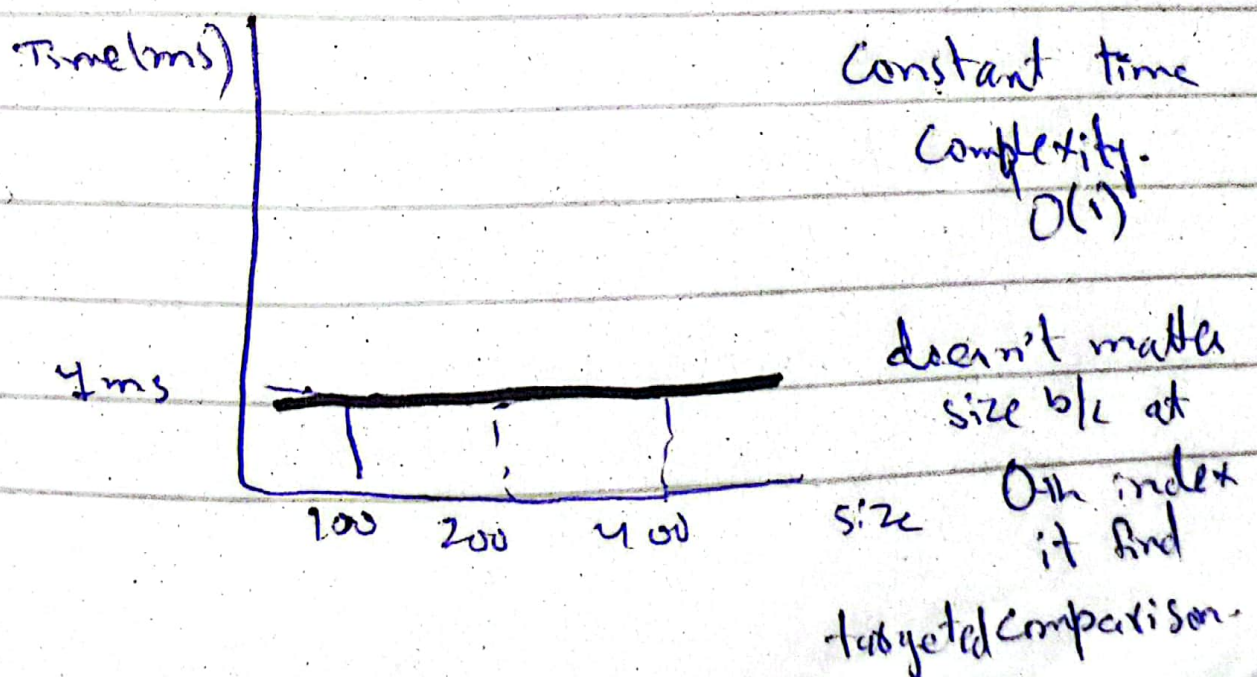
Ex: Size of array = 100 $\Rightarrow$ make 100 comparisons
" = 1 lakh $\Rightarrow$ make 1 lac comparisons
! imagine time taking to make comparisons is 1 lac milli seconds

Graph: Worst case Graph:



That's how time complexity mean as size of input growing, how my time growing.

Best case Graph:



Search in Range

arr = [18, 12, -7, 3, 14, 28]

0	1	2	3	4	5
18	12	-7	3	14	28

Q// Search for 3 in the range of index [1,4]